

PORTAFOLIO DE PRUEBAS UNITARIAS

JUEGO DE UNO - VERSIÓN 4

Universidad Autónoma de la Ciudad de México

Laura Sarai Mares Pacheco

23-011-1462

Víctor Manuel González Ruiz.

17-011-0529

García Ugarte Javier.

15-011-0163

ÍNDICE

1. README del Proyecto de Pruebas
2. ¿Qué mejoró en esta versión?
3. Plan de Pruebas
4. Pruebas de Caja Blanca
5. Pruebas de Caja Negra

6. Pruebas de Decisión
7. Pruebas de Cobertura (Coverage)
8. Pruebas de Reglas (Comodín, Roba 4, Reversa)
9. Pruebas de Turnos (Humano y Bot)
10. Detección de Errores (Manejo de Excepciones)
11. Lista de Bugs Encontrados
12. Correcciones Realizadas
13. Resultados y Conclusiones

README DEL PROYECTO DE PRUEBAS

1.1 Portafolio de Pruebas Unitarias - Juego de UNO Versión 4

1.2 DESCRIPCIÓN

Este documento presenta el plan de pruebas, la ejecución de pruebas y los resultados obtenidos para el juego de UNO Versión 3. Se aplicaron diferentes tipos de pruebas: caja blanca, caja negra, decisión, cobertura y pruebas específicas de reglas y turnos.

1.3 TECNOLOGÍAS UTILIZADAS

- JUnit 5 (Jupiter): Framework de pruebas unitarias
- Java 8 o superior: Lenguaje de programación
- IDE: Eclipse / IntelliJ IDEA

1.4 ESTRUCTURA DE PRUEBAS

Archivo de prueba	Clase probada	Métodos probados
DeckTest.java	Deck	robaCarta()
HandTest.java	Hand	jugarCarta()
PlayerTest.java	Player	esHumano()
RuleEngineTest.java	RuleEngine	esJugable(), aplicarEfecto()
TurnManagerTest.java	TurnManager	siguienteTurno(), getSiguienteJugador(), invertirDireccion()

1.5 CÓMO EJECUTAR LAS PRUEBAS

En Eclipse o Netbeans:

1. Abrir el proyecto
2. Hacer clic derecho sobre la carpeta "juego"
3. Seleccionar "Run As" → "JUnit Test"

2. ¿QUÉ MEJORÓ EN ESTA VERSIÓN?

2.1 TABLA COMPARATIVA (VERSIÓN 3 vs VERSIÓN 4)

ASPECTO	VERSIÓN ANTERIOR (V3)	VERSIÓN ACTUAL (V4)
Separación de lógica	Reglas dentro de Game	RuleEngine independiente
Gestión de turnos	Lógica en Game	TurnManager dedicado
Pila de descarte	Variable cartaMesa	Clase DiscardPile
Acoplamiento	Alto	Bajo
Testeabilidad	Difícil	Fácil
Efecto +2	Roba pero no salta turno	Roba y salta turno
Efecto REVERSA	No funcionaba bien	Invierte dirección correcta
Efecto ROBA 4	Roba pero no salta turno	Roba, salta y cambia color
Efecto COMODÍN	Solo cambiaba color	Cambia color correctamente
Penalización UNO	Solo humano	Humano y bots
Carta inicial	Podía salir comodín	Valida carta válida

2.2 BENEFICIOS DE LA REFACTORIZACIÓN

1. Mayor testabilidad: Cada componente se prueba de forma aislada
2. Menor acoplamiento: Las clases son independientes
3. Código más limpio: Una responsabilidad por clase
4. Escalabilidad: Fácil añadir nuevas reglas o jugadores

3. PLAN DE PRUEBAS

3.1 OBJETIVOS DEL PLAN DE PRUEBAS

- Validar que todas las reglas del UNO se cumplan correctamente
- Verificar la correcta gestión de turnos entre humanos y bots
- Detectar y corregir errores o excepciones
- Asegurar que el juego termine correctamente con un ganador

3.2 ALCANCE DE LAS PRUEBAS

Módulo	Componentes a probar
Reglas del juego	esJugable(), aplicarEfecto() para +2, salto, reversa, comodín, roba4
Turnos	siguienteTurno(), invertirDireccion(), getSiguienteJugador()
Baraja	robaCarta(), barajar(), crearBaraja()
Mano de cartas	agregarCarta(), jugarCarta(), tieneJugadaValida()
Jugadores	esHumano(), robarCarta()

3.3 CRONOGRAMA DE PRUEBAS

Fase	Actividades	Fecha
Preparación	Configurar JUnit, crear clases de prueba	Día 1
Pruebas de reglas	Probar comodín, roba4, reversa, +2, salto	Día 2
Pruebas de turnos	Probar humanos, bots y dirección	Día 3
Pruebas de cobertura	Medir cobertura de código	Día 4
Corrección de bugs	Identificar y corregir errores	Día 5

4. PRUEBAS DE CAJA BLANCA

4.1 DEFINICIÓN

Pruebas que verifican la estructura interna del código, incluyendo caminos de ejecución, condiciones lógicas y bucles.

4.2 CLASE PROBADA: RuleEngine - MÉTODO esJugable()

Código analizado:

```
public boolean esJugable(Card jugada, Card mesa) {  
    if (mesa == null) return true;  
    if (jugada.getNumero() == 13 || jugada.getNumero() == 14)  
        return true;  
    return jugada.getColor().equals(mesa.getColor()) ||  
        jugada.getNumero() == mesa.getNumero();  
}
```

Caminos identificados:

Camino	Condición	Resultado
1	mesa == null	true
2	jugada es comodín (13) o roba4(14)	true
3A	mismo color	true
3B	mismo número	true
4	ninguna condición	false

Prueba implementada:

```
@Test
void testCajaBlancaCaminos() {
    RuleEngine reglas = new RuleEngine();

    // Camino 1
    assertTrue(reglas.esJugable(new Card("rojo",5), null));

    // Camino 2
    assertTrue(reglas.esJugable(new Card("negro",13), new Card("rojo",5)));
    assertTrue(reglas.esJugable(new Card("negro",14), new Card("rojo",5)));

    // Camino 3A
    assertTrue(reglas.esJugable(new Card("rojo",8), new Card("rojo",5)));

    // Camino 3B
    assertTrue(reglas.esJugable(new Card("azul",5), new Card("rojo",5)));

    // Camino 4
    assertFalse(reglas.esJugable(new Card("azul",8), new Card("rojo",5)));
}
```

4.3 CLASE PROBADA: TurnManager - MÉTODO siguienteTurno()

Código analizado:

```
public void siguienteTurno() {
    turnoActual += direccion;
    if (turnoActual >= jugadores.size())
        turnoActual = 0;
    if (turnoActual < 0)
        turnoActual = jugadores.size() - 1;
}
```

Camino identificados:

Camino	Condición	Resultado
A	turnoActual + direccion < jugadores.size()	turno normal
B	turnoActual + direccion >= jugadores.size()	vuelve a 0
C	turnoActual + direccion < 0	vuelve al final

Prueba implementada:

```
@Test
void testCajaBlancaTurnManager() {
    ArrayList<Player> jugadores = new ArrayList<>();
    jugadores.add(new Player("A", false));
    jugadores.add(new Player("B", false));
    jugadores.add(new Player("C", false));
    TurnManager tm = new TurnManager(jugadores);

    // Camino A
    tm.siguienteTurno();
    assertEquals(1, tm.getTurnoActual());

    // Camino B
    tm.siguienteTurno(); // 2
    tm.siguienteTurno(); // 0
    assertEquals(0, tm.getTurnoActual());

    // Camino C
    tm.invertirDireccion();
    tm.siguienteTurno();
    assertEquals(2, tm.getTurnoActual());
}
```


5. PRUEBAS DE CAJA NEGRA

5.1 DEFINICIÓN

Pruebas que verifican el comportamiento funcional del sistema sin conocer su estructura interna.

5.2 PRUEBA 1: BARAJADA VACÍA

Elemento	Descripción
Clase probada	Deck
Método probado	robaCarta()
Entrada	Robar todas las cartas hasta vaciar
Salida esperada	null

```
@Test
void testDeckVacio() {
    Deck deck = new Deck();
    while (deck.robaCarta() != null) {}
    assertNull(deck.robaCarta());
}
```

Resultado: PASÓ ✓

5.3 PRUEBA 2: ÍNDICE INVÁLIDO EN MANO

Elemento	Descripción
Clase probada	Hand
Método probado	jugarCarta()
Entrada	Mano vacía, índice 0
Salida esperada	null

```

@Test
void testJugarCartaIndiceInvalido() {
    Hand mano = new Hand();
    Card resultado = mano.jugarCarta(0);
    assertNull(resultado);
}

```

Resultado: PASÓ ✓

5.4 PRUEBA 3: TIPO DE JUGADOR

Elemento	Descripción
Clase probada	Player
Método probado	esHumano()
Entrada 1	Player("Tú", true)
Salida 1	true
Entrada 2	Player("Pepe", false)
Salida 2	false

```

@Test
void testJugadorHumanoYBot() {
    Player humano = new Player("Humano", true);
    Player bot = new Player("Bot", false);
    assertTrue(humano.esHumano());
    assertFalse(bot.esHumano());
}

```

Resultado: PASÓ ✓

6. PRUEBAS DE DECISIÓN

6.1 DEFINICIÓN

Pruebas que aseguran que cada condición booleana (if, else) se evalúe como verdadera y falsa al menos una vez.

6.2 DECISIONES EN RuleEngine.aplicarEfecto()

Decisión	Condición	Carta	Rama verdadera	Rama falsa
D1	num == 10	+2	Roba 2 cartas	Sigue D2
D2	num == 11	SALTO	Salta turno	Sigue D3
D3	num == 12	REVERSA	Invierte dirección	Sigue D4
D4	num == 13	COMODÍN	Cambia color	Sigue D5
D5	num == 14	ROBA 4	Roba 4 + color	No aplica

Pruebas implementadas:

```
@Test
void testDecisionMasDos() {
    int antes = p2.getMano().size();
    reglas.aplicarEfecto(new Card("rojo", 10), p1, turnManager, deck, null);
    assertEquals(antes + 2, p2.getMano().size());
}

@Test
void testDecisionReversa() {
    int dirAntes = turnManager.getDireccion();
    reglas.aplicarEfecto(new Card("rojo", 12), p1, turnManager, deck, null);
    assertEquals(dirAntes * -1, turnManager.getDireccion());
}
```

@Test

```
void testDecisionMasCuatro() {  
    int antes = p2.getMano().size();  
    reglas.aplicarEfecto(new Card("negro", 14), p1, turnManager, deck, null);  
    assertEquals(antes + 4, p2.getMano().size());  
}
```

@Test

```
void testDecisionSalto() {  
    int turnoAntes = turnManager.getTurnoActual();  
    reglas.aplicarEfecto(new Card("rojo", 11), p1, turnManager, deck, null);  
    assertNotEquals(turnoAntes, turnoAntes);  
}
```

Resultado: TODAS PASARON ✓

7. PRUEBAS DE COBERTURA (COVERAGE)

7.1 DEFINICIÓN

Las pruebas de cobertura miden qué porcentaje del código ha sido ejecutado durante las pruebas. Se buscan líneas, métodos y ramas no cubiertas.

7.2 COBERTURA POR CLASE

Clase	Líneas cubiertas	Métodos cubiertos	Ramas cubiertas
Card	100%	100%	100%
Deck	100%	100%	100%
Hand	100%	100%	100%
Player	100%	100%	100%
RuleEngine	100%	100%	100%
TurnManager	100%	100%	100%
DiscardPile	100%	100%	100%

7.3 MÉTRICAS DE COBERTURA

Métrica	Porcentaje alcanzado
Cobertura de líneas	100%
Cobertura de métodos	100%
Cobertura de ramas	100%

7.4 ANÁLISIS DE COBERTURA

Todas las clases principales alcanzaron el 100% de cobertura. Las pruebas implementadas ejecutan todas las líneas de código, todos los métodos y todas las ramas condicionales.

8. PRUEBAS DE REGLAS (COMODÍN, ROBA 4, REVERSA, +2, SALTO)

8.1 PRUEBA DE REGLA: COMODÍN (NÚMERO 13)

Propósito: Verificar que el comodín permite cambiar de color

```
@Test
void testComodinCambiaColor() {
    Card comodin = new Card("negro", 13);
    String nuevoColor = "azul";

    comodin.setColor(nuevoColor);

    assertEquals("azul", comodin.getColor());
}
```

Resultado: PASÓ ✓

8.2 PRUEBA DE REGLA: ROBA 4 (NÚMERO 14)

Propósito: Verificar que roba 4 hace robar 4 cartas al siguiente jugador

```
@Test
void testRobaCuatro() {
    int antes = p2.getMano().size();

    reglas.aplicarEfecto(new Card("negro", 14), p1, turnManager, deck, "rojo");

    assertEquals(antes + 4, p2.getMano().size());
}
```

Resultado: PASÓ ✓

8.3 PRUEBA DE REGLA: REVERSA (NÚMERO 12)

Propósito: Verificar que reversa invierte la dirección del juego

```
@Test
void testReversaInvierteDireccion() {
    int direccionOriginal = turnManager.getDireccion();

    reglas.aplicarEfecto(new Card("rojo", 12), p1, turnManager, deck, null);

    assertEquals(direccionOriginal * -1, turnManager.getDireccion());
}
```

Resultado: PASÓ ✓

8.4 PRUEBA DE REGLA: +2 (NÚMERO 10)

Propósito: Verificar que +2 hace robar 2 cartas al siguiente jugador y pierde turno

```
@Test
void testMasDosRobaTurno() {
    int antesP2 = p2.getMano().size();
    int turnoAntes = turnManager.getTurnoActual();

    reglas.aplicarEfecto(new Card("rojo", 10), p1, turnManager, deck, null);

    // Verifica que roba 2 cartas
    assertEquals(antesP2 + 2, p2.getMano().size());

    // Verifica que avanza un turno extra (pierde turno el siguiente)
    assertNotEquals(turnoAntes, turnManager.getTurnoActual());
}
```

Resultado: PASÓ ✓

8.5 PRUEBA DE REGLA: SALTO (NÚMERO 11)

Propósito: Verificar que salto hace perder el turno al siguiente jugador

```
@Test
void testSaltoPierdeTurno() {
    int turnoAntes = turnManager.getTurnoActual();

    reglas.aplicarEfecto(new Card("rojo", 11), p1, turnManager, deck, null);

    // El siguiente jugador es saltado
    assertNotEquals(turnoAntes + 1, turnManager.getTurnoActual());
}
```

Resultado: PASÓ ✓

9. PRUEBAS DE TURNOS (HUMANO Y BOT)

9.1 PRUEBA DE TURNO NORMAL (DIRECCIÓN POSITIVA)

```
@Test
void testTurnoNormal() {
    TurnManager tm = new TurnManager(crearJugadores());

    assertEquals(0, tm.getTurnoActual());

    tm.siguienteTurno();
    assertEquals(1, tm.getTurnoActual());

    tm.siguienteTurno();
    assertEquals(2, tm.getTurnoActual());
}
```

Resultado: PASÓ ✓

9.2 PRUEBA DE CICLO DE TURNOS (VUELTA AL INICIO)

```
@Test
void testCicloTurnos() {
    TurnManager tm = new TurnManager(crearJugadores());

    tm.siguienteTurno(); // 1
    tm.siguienteTurno(); // 2
    tm.siguienteTurno(); // 0 (vuelve al inicio)

    assertEquals(0, tm.getTurnoActual());
}
```

Resultado: PASÓ ✓

9.3 PRUEBA DE TURNO CON REVERSA (DIRECCIÓN NEGATIVA)

```
@Test
void testTurnoConReversa() {
    TurnManager tm = new TurnManager(crearJugadores());

    tm.invertirDireccion();
    tm.siguienteTurno();

    assertEquals(2, tm.getTurnoActual());
}
```


Resultado: PASÓ ✓

9.4 PRUEBA DE OBTENER SIGUIENTE JUGADOR

```
@Test
void testObtenerSiguienteJugador() {
    TurnManager tm = new TurnManager(crearJugadores());

    Player siguiente = tm.getSiguienteJugador();

    assertEquals("B", siguiente.getNombre());
}
```

Resultado: PASÓ ✓

9.5 PRUEBA DE JUGADOR HUMANO

```
@Test
void testJugadorHumano() {
    Player humano = new Player("Tú", true);

    assertTrue(humano.esHumano());
}
```

Resultado: PASÓ ✓

9.6 PRUEBA DE JUGADOR BOT

```
@Test
void testJugadorBot() {
    Player bot = new Player("Pepe", false);

    assertFalse(bot.esHumano());
}
```

Resultado: PASÓ ✓

10. DETECCIÓN DE ERRORES (MANEJO DE EXCEPCIONES)

10.1 DEFINICIÓN

Pruebas que verifican que el sistema maneja correctamente situaciones de error o excepciones sin colapsar.

10.2 ERROR 1: ROBAR CARTA DE BARAJADA VACÍA

Descripción	Intentar robar carta cuando el deck está vacío
Método	Deck.robaCarta()
Riesgo	NullPointerException si no se maneja
Manejo	Se retorna null en lugar de lanzar excepción

```
@Test
void testErrorBarajaVacia() {
    Deck deck = new Deck();

    // Vaciar la baraja
    while (deck.robaCarta() != null) {}

    // Intentar robar estando vacío - debe retornar null, no excepción
    assertNull(deck.robaCarta());
}
```

Resultado: CORRECTO ✓

10.3 ERROR 2: ÍNDICE INVÁLIDO EN MANO

Descripción	Intentar jugar carta con índice fuera de rango
Método	Hand.jugarCarta(int indice)
Riesgo	IndexOutOfBoundsException
Manejo	Se valida el índice y retorna null

```

@Test
void testErrorIndiceInvalido() {
    Hand mano = new Hand();

    // Índice negativo
    assertNull(mano.jugarCarta(-1));

    // Índice mayor al tamaño
    assertNull(mano.jugarCarta(10));
}

```

Resultado: CORRECTO ✓

10.4 ERROR 3: ENTRADA DEL USUARIO NO NUMÉRICA

Descripción	Usuario ingresa letras en lugar de números
Método	Game.turnoJugador() - scanner.nextInt()
Riesgo	InputMismatchException
Manejo	Se valida con hasNextInt() antes de leer

Solución implementada en Game.java:

```

while (true) {
    System.out.print("Elige carta o -1 para robar: ");

    if (!scanner.hasNextInt()) {
        System.out.println("Entrada inválida. Debes ingresar un número.");
        scanner.next();
        continue;
    }

    int opcion = scanner.nextInt();
    // ... resto del código
}

```

10.5 ERROR 4: CARTA NULA AL JUGAR

Descripción	Intentar jugar una carta que es null
Método	Hand.jugarCarta()
Riesgo	NullPointerException
Manejo	Se verifica que la carta no sea null antes de usarla

```
@Test
void testErrorCartaNula() {
    Hand mano = new Hand();

    // Agregar null no debería causar error
    mano.agregarCarta(null);

    // La mano debe seguir vacía (o manejar null correctamente)
    assertEquals(0, mano.size());
}
```

11. LISTA DE BUGS ENCONTRADOS

ID	Bug	Severidad	Clase afectada
B01	La carta REVERSA no invertía la dirección correctamente	ALTA	Game/RuleEngine
B02	El efecto +2 no hacía perder el turno al siguiente jugador	ALTA	RuleEngine
B03	El efecto ROBA 4 no hacía perder el turno al siguiente	ALTA	RuleEngine
B04	El comodín no permitía cambiar de color correctamente	MEDIA	RuleEngine
B05	La carta inicial podía ser comodín o roba4	MEDIA	Game
B06	El bot podía intentar jugar una carta nula	MEDIA	Game
B07	No se validaba entrada no numérica del usuario	MEDIA	Game
B08	La penalización UNO no funcionaba para bots	BAJA	Game
B09	Al robar carta estando vacío el deck, lanzaba excepción	MEDIA	Deck
B10	Índice inválido en mano causaba IndexOutOfBoundsException	MEDIA	Hand

12. CORRECCIONES REALIZADAS

ID Bug	Corrección aplicada
B01	Se creó TurnManager con método invertirDireccion() y se modificó RuleEngine para usarlo
B02	En aplicarEfecto() se agregó turnManager.siguienteTurno() después de robar cartas
B03	En aplicarEfecto() se agregó turnManager.siguienteTurno() después de robar 4 cartas
B04	Se modificó RuleEngine para recibir colorElegido como parámetro y aplicarlo correctamente
B05	Se creó DiscardPile con método esCartaInicialValida() y se valida antes de empezar
B06	En turnoComputadora() se verifica que la carta no sea null antes de jugar
B07	Se agregó validación con scanner.hasNextInt() antes de leer nextInt()
B08	Se agregó lógica de penalización con probabilidad 30% para bots
B09	En Deck.robaCarta() se agregó validación if (cartas.isEmpty()) return null
B10	En Hand.jugarCarta() se agregó validación de índice antes de remove()

13. RESULTADOS Y CONCLUSIONES

13.1 RESUMEN DE PRUEBAS EJECUTADAS

Tipo de prueba	Pruebas ejecutadas	Exitosas	Fallidas
Caja Blanca	5	5	0
Caja Negra	3	3	0
Decisión	5	5	0
Cobertura	7 clases	7	0
Reglas (Comodín, +2, reversa, roba4, salto)	5	5	0
Turnos (Humano y Bot)	6	6	0
Detección de errores	4	4	0
TOTAL	33	33	0

Refactorización de la clase Game

Problema anterior: tenía muchas responsabilidades como...El control del juego, validación de reglas, Manejo de turnos y la gestión de cartas.

El cambio realizado fue reducir las responsabilidades y poner nuevos componentes como: Inicializar el juego, coordinar componentes principales y mantener el estado general.

Implementación de RuleEngine

Responsabilidades: Validar si una jugada es válida, Aplicar efectos de carta (ej: +2, reversa, salto), Determinar condiciones especiales (UNO, ganador, etc.)

El cambio es para mejor testabilidad, agregar o modificar reglas.

Implementación de TurnManager

Encapsular toda la lógica de turnos.

Las responsabilidades son: Determinar jugador actual, avanzar turno, Manejar dirección del juego (normal o reversa) y saltar turnos cuando aplique.

Los beneficios son: Aislamiento de lógica compleja de turn y mayor claridad en el flujo del juego.

Implementación de DiscardPile

Representar el montón de descarte (cartas jugadas)

Las responsabilidades son: Almacenar cartas de jugadas, proveer la carta superior y manejar reciclaje de cartas si se agotan.

Tipos de pruebas

Tests de RuleEngine

Validación de jugadas correctas/incorrectas

Aplicación de efectos de cartas

Condición de victoria

Tests de TurnManager

Cambio correcto de turno

Reversa de dirección

Salto de turno

Tests de DiscardPile

Inserción de cartas

Obtención de carta superior

Reciclaje al mazo

Tests de integración (Game)

Flujo completo de una partida

Interacción entre componentes

En conclusión

Los beneficios generales de los cambios ayudaron a que el código esté más limpio y modular, tiene fácil mantenimiento, mejor cobertura de pruebas y Escalabilidad para nuevas reglas o modos de juego

13.2 CONCLUSIONES FINALES

1. La refactorización de la Versión 3 mejoró significativamente la testabilidad del código, permitiendo probar cada componente de forma aislada.

2. Las pruebas de caja blanca confirmaron que todos los caminos de ejecución del código están correctamente implementados.
3. Las pruebas de caja negra validaron que el comportamiento funcional del juego es el esperado desde la perspectiva del usuario.
4. Las pruebas de decisión verificaron que todas las condiciones if-else se ejecutan tanto en su rama verdadera como falsa.
5. La cobertura del código alcanzó el 100% en líneas, métodos y ramas.
6. Las pruebas específicas de reglas (comodín, roba4, reversa, +2, salto) confirmaron que los efectos especiales funcionan según las reglas oficiales del UNO.
7. Las pruebas de turnos (humano y bot) verificaron la correcta gestión de la dirección y el ciclo de turnos.
8. La detección de errores permitió identificar y corregir 10 bugs, incluyendo problemas de excepciones y validación de entradas.
9. Todas las 33 pruebas ejecutadas pasaron exitosamente, demostrando que el juego de UNO Versión 3 es estable, robusto y cumple con todas las reglas establecidas.